The CDOClient code can be found at CETIC's gitlab (http://git.cetic.be/paasage/cdo_client). This code can be used to connect to a CDOServer through a CDOSession and then make queries, store models or export models / objects returned from queries. It also includes the domain code of CAMEL and WP3 meta-models, although this code can of course be exploited in a different way through using different, forthcoming projects in gitlab (one for CAMEL and another for WP3 meta-models). The main idea of including the domain code was that it had to be shown through the main method of the CDOClient the main functionality exposed, which includes creating a Cerif model and storing it in the CDOServer.

The CDOClient code can be used in two alternative ways, where the respective executable files can be created via "mvn clean install" at the main directory of the CDOClient code:

- for testing purposes to see that the main functionality exposed is correct. To this end, the main method includes a specific exploitation scenario which can be checked against a running CDOServer. Thus, the user just have to call the main method, which can be performed in two alternative ways: (1) "mvn exec:java" and (2) "java -jar client-1.1-jar-with-dependencies.jar" (in second case please check that the file paths provided are correct for files to be used as input or produced as output from the CDOClient method calls).
- for real exploitation through creating a CDOClient inside the main user code and exploiting its functionality to perform some tasks. The user will have to include in the libraries of its code, the jar file "client-1.1-jar-with-dependencies.jar". In case of maven projects, just a new dependency has to be created in the respective pom.xml file.

For real exploitation, the following exploitation scenarios are relevant:

1. Load a model from a XMI file and store it in the CDOServer. This can be useful in case the user has produced through an Eclipse editor a particular XMI file and desires to store it in the CDOServer such that it can be exploited later on by the respective PaaSage components. It could also be useful in case that there is one PaaSage component that produces a XMI file and another one that desires to load it, process it and then store it in the CDOServer.
2. Create a model programmatically and then store the model in the CDOServer, where there can be also the optional choice of exporting the model as a XMI file. This can be of course useful in case a PaaSage component desires to create a particular model and store it in the CDOServer in order to be exploited by another PaaSage component, where CDOServer can be actually consider an indirect communication mean between these two components.
3. Make a query to the CDOServer in order to find and load a particular information and then either change this information and store it back to CDOServer, create a model out of it and then store it in the CDOServer or just export the query result. This more complicated scenario can be useful for the following cases: (a) the user desires to modify a model. Thus, he/she has first to perform a query in order to obtain it and then modify it and store it back to the CDOServer. Imagine, for instance, the case of a feature model of a particular provider that has to be modified when a new cloud service is offered by this provider; (b) the user desires to create a model out of an existing model or a part of it. Thus, he/she has to issue a query and then browse the results in order to create the corresponding model to be stored in the CDOServer. Imagine, for instance, the case where we have to associate the cerif information from a cloud provider with the feature model that this provider offers. When the cerif information is created, we will have to obtain the feature model of the cloud provider and then make the appropriate association between the instance of CloudProvider and the instance of the FeatureModel (see property providerModel in the organisation subpackage of CAMEL meta-model); (c) the user wants to view a particular output from the PaaSage platform. In this case, a particular platform code will have to issue the query and export the result as an XMI file which can then be processed by the visualization code in order to produce the respective

visualization (e.g., imagine an Eclipse editor which is exploited by the user to see the result or a fixed visualization code that takes as input a XMI file and it is not desired to modify it in order to make the query itself and process the respective results in memory).

4. Make a query in order to find a particular object to be removed from the CDOServer. This scenario can include the repetition of the above two actions many times until all desired objects are removed. Please be aware that the respective methods used to realize the scenario should be used with care as they do not anticipate the following situation: when the object is deleted from its containment or non-containment association, the respective object to which it is associated may no longer be valid. For instance, if the object to be deleted is associated with a one-to-one relation with its associating object, then by removing it the associating object becomes invalid. As another example, if there is an one-to-many association which should not be empty, then if the deleted object is the last one in the association, this will make its associating object invalid.

The CDOClient code comprises just one main class called "CDOClient" which can be exploited to realize the above exploitation scenarios. This class comprises the following methods:

- *Name*: **registerPackage**
  - *Description*: This method is used for registering an EPackage that maps to the domain code to be manipulated. We take the view that the user is responsible for registering any EPackage that does not correspond to those mapping to the CAMEL and WP3 meta-models, as the system will not be aware of a new meta-model and the respective domain code.
  - *Input Parameters*:
    - **pack** : The *EPackage* object to be registered.
  - *Output*: No output needs to be produced
  - *Usage*: By supposing that a *CDOClient* instance *cl* has been created and that the user desires to register the CAMEL package, then the following code will have to be produced:
    cl.registerPackage(CamelPackage.eINSTANCE);
- *Name*: **openTransaction**
  - *Description*: This method is used to open a CDO transaction and return it to the user. Then, the user can issue queries or store models through the direct exploitation of this transaction (object) instead of using the corresponding methods exposed by the CDOClient class. It must be noted that once all user tasks are ended, the CDO transaction created needs to be closed by calling the closeTransaction method with the transaction object as input parameter.
  - *Input Parameters*:
    - None
  - *Output*: An object of class *CDOTransaction*
  - *Usage*: By supposing that a *CDOClient* instance *cl* has been created, the user can obtain the CDO transaction created as follows: CDOTransaction trans = cl.openTransaction();

- *Name*: **openView**
  - *Description*: This method is used to open a CDO view and return it to the user. Then, the user can issue queries through the direct exploitation of this view (object) instead of using the corresponding methods exposed by the CDOClient class. It must be noted that once all corresponding user tasks are ended, the CDO view created needs to be closed by calling the closeView method with the view object as input parameter.
  - *Input Parameters*:
    - None
  - *Output*: An object of class *CDOView*
  - *Usage*: By supposing that a *CDOClient* instance *cl* has been created, the user can obtain the CDO view created as follows: CDOView view = cl.openView();
- *Name*: **closeTransaction**
  - *Description*: This method is used to close a previously opened CDO transaction, obviously after all corresponding user tasks have been executed.
  - *Input Parameters*:
    - **trans** : The *CDOTransaction* object to be closed.
  - *Output*: None
  - *Usage*: By supposing that a *CDOClient* instance *cl* has been created and a *CDOTransaction trans* has been opened, the user can close the CDO transaction created as follows: CDOTransaction trans = cl.closeTransaction(trans);
- *Name*: **closeView**
  - *Description*: This method is used to close a previously opened CDO view, obviously after all corresponding user tasks have been executed.
  - *Input Parameters*:
    - **view** : The *CDOView* object to be closed.
  - *Output*: None
  - *Usage*: By supposing that a *CDOClient* instance *cl* has been created and a *CDOView view* has been opened, the user can close the CDO view created as follows: CDOView view = cl.closeView(view);
- *Name*: **storeModel**
  - *Description*: This method is used to store a model in the CDO Server under a CDOResource with a particular name. Please note that no transaction needs to be created for performing this storage as this is conducted internally by the method.

- *Input Parameters*:
  - **model** : The model to be stored as an instance of *EObject* class (which is indeed the case as all models created are instances of classes that are subclasses of EObject).
  - **resourceName** : The name of the CDOResource as a String under which the model will be stored.
- *Output*: None
- *Usage*: By supposing that a *CDOClient* instance *cl* has been created and a *EObject model* has been created (programmatically or via calling the loadModel method of CDOClient class), the user can store this model under a CDOResource named as "camel1" as follows: cl.storeModel(model,"camel1");

- *Name*: **loadModel**
  - *Description*: This method is used to load a model from a XMI file which is situated in the directory whose path is designated by the PAASAGE_CONFIG_DIR system variable. Obviously, then the model can be read, processed and possibly stored in the CDO Server.
  - *Input Parameters*:
    - **pathName** : The path (as a String) to the XMI file containing the resource/model to load
  - *Output*: The model as instance of EObject class
  - *Usage*: By supposing that a *CDOClient* instance *cl* has been created and that the file containing the model to be loaded is named as "model.xmi" situated in the current directory "input" sub-directory, then the user can load the desired model as follows: EObject model = cl.loadModel("input/model.xmi");

- *Name*: **createCerifModel**
  - *Description*: This method is used to create (programmatically) a cerif model (instance of OrganisationModel class) which can then be stored in the CDO Server by calling the storeModel method.
  - *Input Parameters*:
    - None
  - *Output*: The model as an instance of EObject
  - *Usage*: By supposing that a *CDOClient* instance *cl* has been created, you can create and obtain the cerif model as follows: EObject model = cl.createCerifModel();

- *Name*: **exportModel** (overloaded with two method instances - the **first** is described here)
  - *Description*: This method is used to export a model that has been stored under a particular CDOResource into a XMI file with a particular path. We should highlight that if the required model is not placed at the root of the CDOResource designated by the user (which

actually maps to a CamelModel), then we consider the first model belonging to the CamelModel of the same type as the one requested by the user to be exported (e.g, the first DeploymentModel belonging to the CamelModel referenced by exploiting the containment association *deploymentModels*). If a different model of the same type is desired, then it is better to exploit the second instance of the method after the desired model has been obtained through a query. We must also note that the user is responsible for providing correct input parameters as well as ensuring that the requested model has been stored under the designated CDOResource.

- *Input Parameters*:
  - **resourceName** : The name (as a String) of the CDOResource under which the model has been stored.
  - **c** : The class (java *Class* object) which is instantiated by the required model to be stored (e.g., DeploymentModel.class).
  - **filePath** : The path to the file (as a String) on which the required model will be stored.
- *Output*: None
- *Usage*: By supposing that a *CDOClient* instance *cl* has been created, the file to be used for storing the model has the relative path "output/model.xmi", the name of the CDOResource is "camel1" and a model of type DeploymentModel needs to be stored, then the exporting will be performed as follows:
  cl.exportModel("camel1",DeploymentModel.class,"output/model.xmi");

- *Name*: **exportModel** (overloaded with two method instances - the **second** is described here)
  - *Description*: This method is used to export a model, which is provided directly as input parameter, into a XMI file with a particular name. The model could be created programmatically or retrieved by issuing a query. We must highlight here that the method should be called after a CDOTransaction has been created, as otherwise an Exception will be thrown (this is a restriction from CDO which requires that a transaction must have been opened in order to be able to create a resource out of a (cdo) model and store it).
  - *Input Parameters*:
    - **model** : The model to be stored as instance of *EObject* class.
    - **filePath** : The path to the file (as a String) on which the required model will be stored.
  - *Output*: None
  - *Usage*: By supposing that a *CDOClient* instance *cl* has been created, the file to be used for storing the model has the relative path "output/model.xmi", and a *model* of type DeploymentModel has been created or fetched, then the exporting will be performed as follows: cl.exportModel(model, "output/model.xmi");

- *Name*: **runQuery**
  - *Description*: This method is used to run a query over a CDO Server / Repository and obtain

back a list of instances of EObject class. Three types of query dialect are supported: OCL, SQL and HQL. Through this method, the user has also the option to export the first object from the list to be returned into a XMI file.

- *Input Parameters*:
  - **dialect** : The dialect of the query as a String with three possible values: "OCL", "SQL" and "HQL".
  - **queryStr** : The actual query as a String which obviously should conform to the dialect given.
  - **fileName** : The name of the file (as a String) on which the first instance of *EObject* class (i.e., first query result) will be stored. The file will be situated under the directory designated by the PAASAGE_CONFIG_DIR system variable.
- *Output*: None
- *Usage*: By supposing that a *CDOClient* instance *cl* has been created, the query dialect is HQL, the desired query is "`select dm from DeploymentModel dm`" and the file to be used for storing the first query result is named as "queryResult.xmi", then the query issuance will be performed as follows: List<EObject> results = cl.runQuery("HQL","select dm from DeploymentModel dm", "queryResult.xmi");

- *Name*: **deleteObject (**overloaded with two method instances - the **first** is described here**)**
  - *Description*: This method is used to delete an object from the CDO Server / Repository based on its CDOID provided as input.
  - *Input Parameters*:
    - **cdoID** : The CDOID of the object to be deleted.
  - *Output*: None
  - *Usage*: By supposing that a *CDOClient* instance *cl* has been created and that a query has been issued to obtain the object *obj* to be deleted, then the object deletion can be performed as follows: cl.deleteObject(obj.cdoID());

- *Name*: **deleteObject (**overloaded with two method instances - the **second** is described here**)**
  - *Description*: This method is used to delete an object from the CDO Server / Repository by also exploiting the transaction that is given as input.
  - *Input Parameters*:
    - **object** : The CDOObject to be deleted.
    - **trans** : The CDOTransaction under which the deletion will occur.
    - **commitAndClose** : A boolean parameter indicating whether the transaction given as input will be committed and closed or not. In the latter case, the user is responsible for

either committing and closing the transaction him/herself or providing the last call of this method with the value of true for this parameter.

- *Output*: None
- *Usage*: By supposing that a *CDOClient* instance *cl* has been created and that a query has been issued via transaction *trans* to obtain the object *obj* to be deleted, then the object deletion which should be immediately committed can be performed as follows: cl.deleteObject(obj, trans, true);

- *Name*: **closeSession**
  - *Description*: This method is used to close the CDOSession that is opened when a CDOClient object is created. Thus, this method should be the very last one from those that will be exploited for a particular user task / exploitation scenarion.
  - *Input Parameters*: None
  - *Output*: None
  - *Usage*: By supposing that a *CDOClient* instance *cl* has been created and used/exploited, the very last command to be executed will be as follows: cl.closeSession();

Now, we are going to show code fragments that realize the three exploitation scenarios aforementioned.

Exploitation Scenario 1:

```
//Create CDOClient object
CDOClient cl = new CDOClient();

//Load model from XMI file
EObject model = cl.loadModel("Sens_App_Scenario.xmi");

//Store model into a CDOResource named "SensApp"
cl.storeModel(model,"SensApp");

//Close CDO Session
cl.closeSession();
```

Exploitation Scenario 2:

```
//Create CDOClient object
CDOClient cl = new CDOClient();

//Create model programmatically - we use the already supplied method of CDOClient to create
//a cerif model but the user can of course call another method of his/her own which will create
//the desired model programmatically, e.g., EObject model = createMyModel();
EObject model = cl.createCerifModel();
```

```
//Store model into a CDOResource named "cerif"
cl.storeModel(model,"cerif1");

//Also export model to visually inspect it
cl.exportModel(model,"output/cerif1.xmi");

//Close CDO Session
cl.closeSession();
```

Exploitation Scenario 3:

```
//Create CDOClient object
CDOClient cl = new CDOClient();

//Open a  CDOTransaction
Transaction trans = cl.openTransaction();

//Obtain all objects under a specific CDOResource named as "camel1"
List<EObject> results = trans.getResource("camel1").getContents();

//Process each object
for (EObject obj: results){
System.out.println("Got object: " + obj);
//do sth with the object
......
}

//Export one of the results to visually inspect it
cl.exportModel(results.get(2),"queryResult.xmi");

//Close the opened transaction
cl.closeTransaction(trans);

//Close CDO Session
cl.closeSession();
```

Exploitation Scenario 4 (showing two alternative ways to delete objects):

```
//Create CDOClient object
CDOClient cl = new CDOClient();

//1. Beginning of first deletion alternative
//Open a  CDOTransaction
Transaction trans = cl.openTransaction();

//Obtain object to be deleted
```

```
ExternalIdentifier id = trans.createQuery("hql","select id from ExternalIdentifier id where
id.name='ID3'").getResult(ExternalIdentifier.class).get(0);

//Delete Object as well as commit and close transaction
cl.deleteObject(id, trans, true);

//2. Beginning of second deletion alternative
//Open a CDO view
CDOView view = cl.openView();

//Get object to be deleted
id = view.createQuery("hql","select id from ExternalIdentifier id where
id.name='ID2'").getResult(ExternalIdentifier.class).get(0);

//Delete Object
cl.deleteObject(id.cdoID());

//Close CDO View
cl.closeView(view);

//Close CDO Session
cl.closeSession();
```

For inquiries about the CDOClient code, please contact Kyriakos Kritikos via the following email address: kritikos AT ics.forth.gr.